# Secure Employee Document Vault

## Secure Document Management System using Flask and AWS Cloud Services

Python Flask | AWS Secrets Manager | AWS KMS

Amazon S3 | Amazon DynamoDB | Boto3

**Document Type**

Technical Design Documentation

**Version**

1.0.0 | June 2026

**Classification**

Confidential

Built with AWS Cloud Services | Flask Python Framework | Enterprise Security Standards

# Table of Contents

## 1 Executive Summary

The **Secure Employee Document Vault** is an enterprise-grade, cloud-native document management system built with Python Flask and Amazon Web Services (AWS). It provides organizations with a robust, secure, and scalable platform for storing, managing, retrieving, and auditing sensitive employee documents throughout their entire lifecycle.

In today's digital-first workplace, organizations handle large volumes of highly sensitive employee documentation — including salary slips, offer letters, employment contracts, appraisal reports, and identity documents. Managing this data securely while ensuring compliance, confidentiality, and operational efficiency presents a significant technical challenge.

| Pillar | Implementation | Benefit |
|---|---|---|
| **Security** | AWS KMS Encryption + Secrets Manager | Zero hardcoded credentials |
| **Encryption** | AES-256 via AWS KMS | Data secured at rest and in transit |
| **Secret Mgmt** | AWS Secrets Manager | Centralized config, rotation support |
| **Cloud-Native** | S3 + DynamoDB + KMS | Scalable, managed infrastructure |
| **Access Control** | Flask route protection | Controlled document access |

# 2 Business Problem

Organizations across industries regularly handle sensitive employee documentation. Without a robust, purpose-built document management system, these organizations face significant operational and security risks.

## 2.1 Categories of Sensitive Employee Documents

| Document Type | Sensitivity Reason |
|---|---|
| ■ Salary Slips | Monthly payroll details, tax deductions, net pay — highly confidential financial data |
| ■ Offer Letters | Salary packages, joining dates, designation details — competitive intelligence |
| ■ Employment Contracts | Terms of employment, non-compete clauses, intellectual property agreements |
| ■ Appraisal Reports | Performance ratings, manager feedback, increment history |
| ■ Identity Documents | Aadhaar, PAN, Passport copies — personal identifiable information (PII) |

## 2.2 Traditional Insecure Approach

Most organizations rely on ad-hoc, insecure approaches to document storage that create multiple points of vulnerability. The following diagram illustrates the typical insecure architecture:

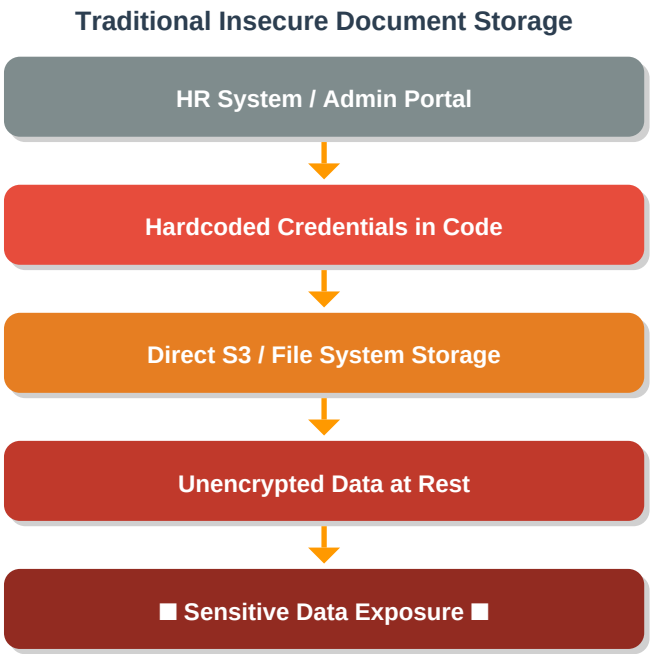

*Figure 2.1 — Traditional Insecure Document Storage Architecture*

## 2.3 Core Problems Identified

✗ **Data Leakage Risk:** Files stored without encryption can be exposed through misconfigured S3 bucket policies, insider threats, or unauthorized access

✗ **Hardcoded Secrets:** AWS credentials, bucket names, and KMS key IDs embedded directly in source code create critical security vulnerabilities

✗ **Unencrypted Storage:** Documents stored in plain text or with only transport-layer encryption remain vulnerable if storage is compromised

✗ **No Centralized Management:** Scattered configuration across environments makes secret rotation, auditing, and compliance reporting nearly impossible

✗ **No Metadata Tracking:** Without structured metadata, searching, auditing, and managing document lifecycles becomes manual and error-prone

# 3 Proposed Solution

The Secure Employee Document Vault addresses each identified problem with a purpose-built, cloud-native architecture that implements AWS security best practices. Every component is selected to eliminate the vulnerabilities present in traditional approaches.

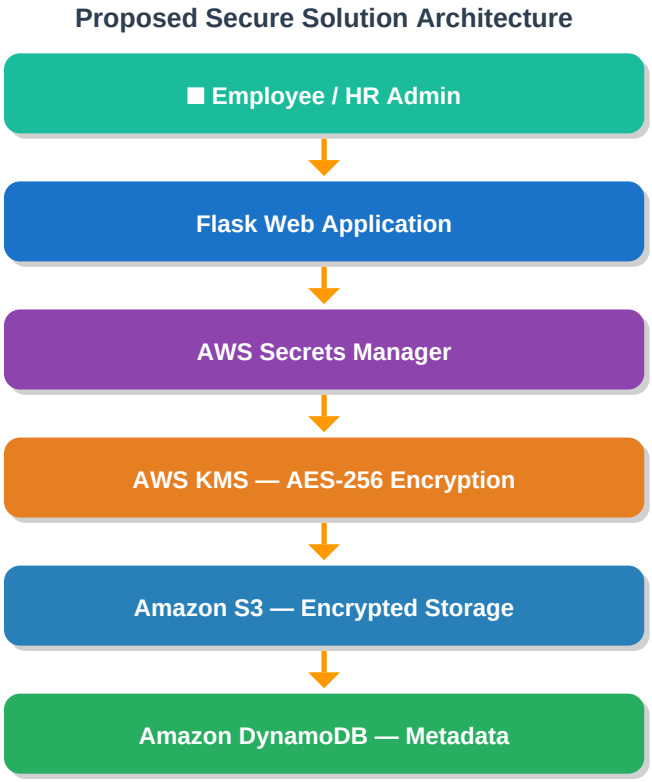


*Figure 3.1 — Proposed Secure Document Vault Solution Flow*

## 3.1 How Each Problem is Solved

✓ **Data Leakage:** All files encrypted with AES-256 via AWS KMS before storage. Even S3 bucket access does not expose plaintext data.

✓ **No Hardcoded Secrets:** All configuration (bucket name, KMS key ID, table name) stored in AWS Secrets Manager. Retrieved at runtime.

✓ **Encryption:** AWS KMS provides envelope-ready encryption. Files stored with .enc extension indicating encrypted state.

✓ **Centralized Management:** Single Secrets Manager secret controls all application configuration. Rotation supported natively.

✓ **Metadata Tracking:** DynamoDB stores employee_id, file_name, document_type, and upload_time for every document.

# 4 System Architecture

The system is built entirely on AWS managed services, ensuring high availability, automatic scaling, and built-in security features. The architecture follows AWS Well-Architected Framework principles with emphasis on the Security pillar.

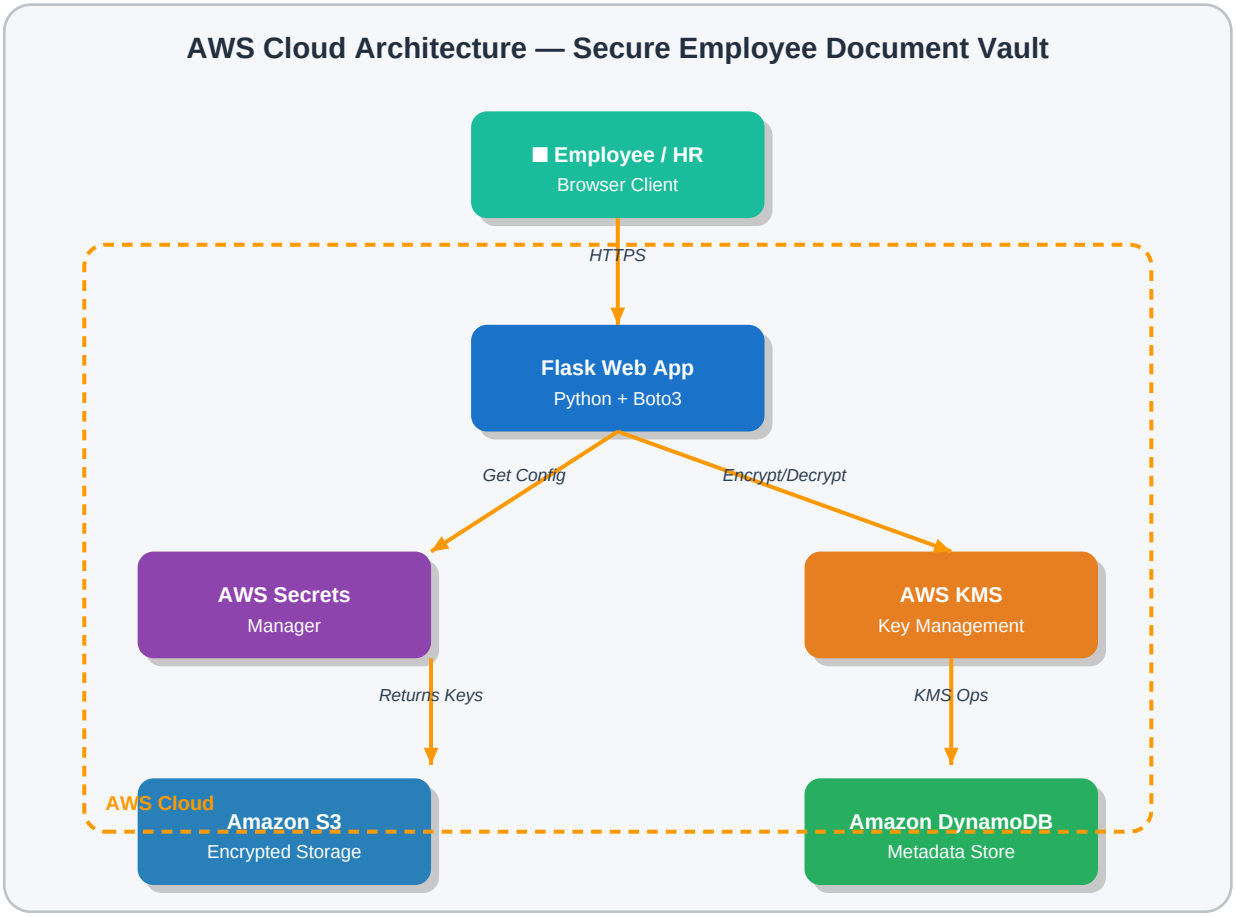


*Figure 4.1 — Complete AWS Architecture Diagram*

## 4.1 Component Roles

### Flask Web Application

Python web framework serving as the application layer. Handles HTTP requests, orchestrates service calls, and renders HTML templates. Acts as the single entry point for all user interactions.

### AWS Secrets Manager

Centralized secrets storage service. Stores application configuration (bucket name, KMS key ID, DynamoDB table name) as a JSON secret. Retrieved at runtime via Boto3 API calls.

### AWS KMS (Key Management Service)

Managed cryptographic service providing AES-256 encryption. Used to encrypt file contents before S3 upload and decrypt after download. Keys are never exposed to application code.

### Amazon S3 (Simple Storage Service)

Object storage for encrypted document files. Each file stored under a path structured as employee_id/filename.enc, providing logical organization by employee.

### Amazon DynamoDB

NoSQL database storing document metadata. Primary key: employee_id (partition), file_name (sort). Enables fast, serverless querying without managing database infrastructure.

# 5 AWS Services Used

## 5.1 AWS Secrets Manager

AWS Secrets Manager is a secrets management service that enables secure storage, retrieval, and rotation of application secrets. In this application, it replaces all hardcoded configuration values with a single, centrally managed secret.

### Secret Structure

```
{
    "bucket_name":  "arun-secure-files-demo-2026",
    "kms_key_id":   "620841a8-a83a-4efc-b950-fe07857c485f",
    "table_name":   "employee-documents"
}
```

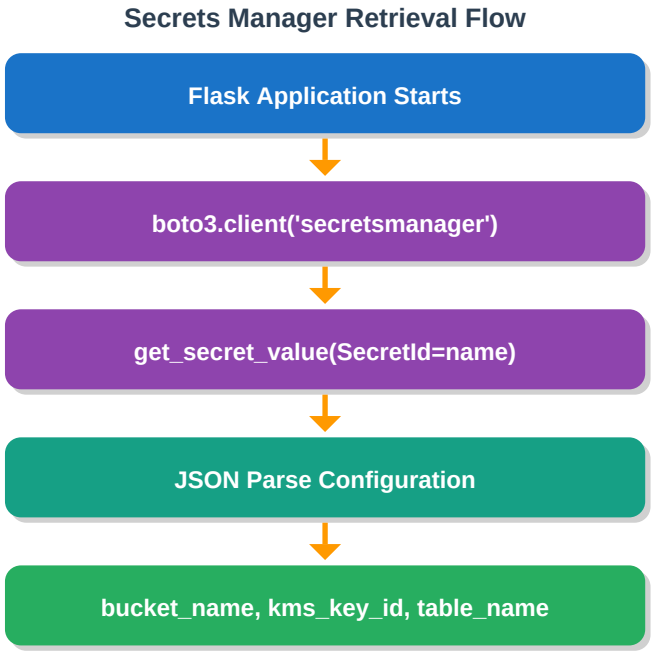


*Figure 5.1 — Secrets Manager Configuration Retrieval*

### Key Benefits

- No hardcoded credentials in source code or environment variables
- Secret values never appear in application logs or git history
- Supports automatic secret rotation without application restart
- Fine-grained IAM policies control which services can access secrets
- Full audit trail via AWS CloudTrail

## 5.2 AWS KMS — Key Management Service

AWS KMS provides hardware-backed cryptographic key management. The application uses KMS for symmetric AES-256-GCM encryption of all document files before they are uploaded to S3, and decryption when files are downloaded.

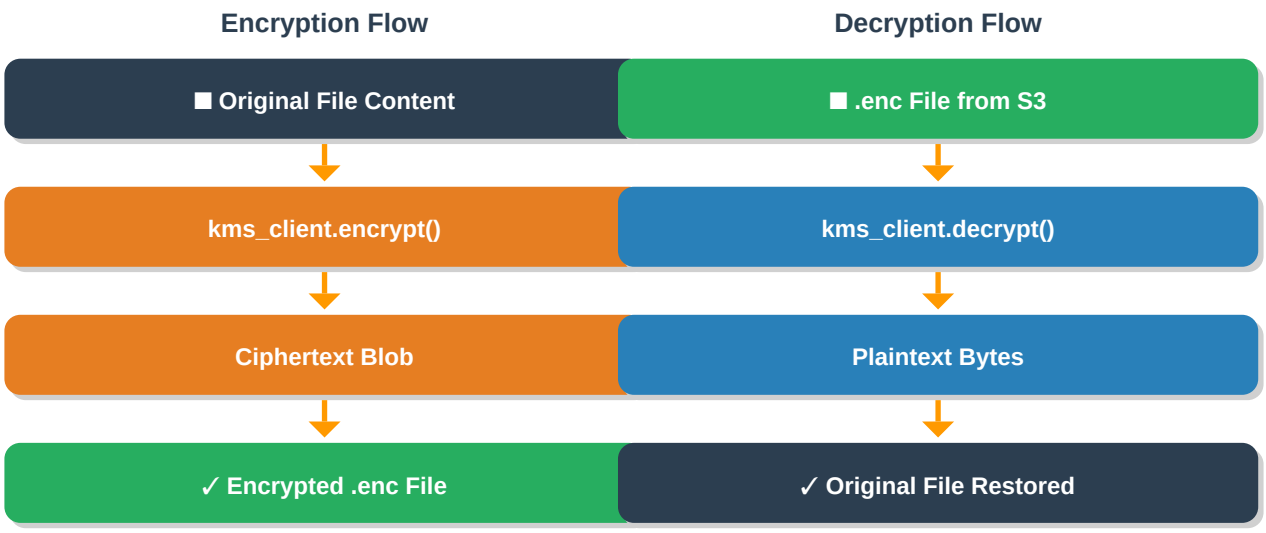


*Figure 5.2 — KMS Encrypt and Decrypt Operations*

### Why AWS KMS?

- Hardware Security Module (HSM) backed — keys never leave AWS infrastructure
- Integrated with IAM for fine-grained access control
- Automatic key rotation capability
- Pay-per-API-call pricing — cost-effective for document operations
- Full audit logging via AWS CloudTrail

## 5.3 Amazon S3 — Encrypted Document Storage

Amazon S3 stores all encrypted document files. Documents are organized using a logical key structure that groups files by employee ID, enabling efficient listing and retrieval.

### S3 Key Structure

```
Bucket: arun-secure-files-demo-2026
■■■ EMP001/
■   ■■■ salary_report_jan2026.txt.enc
■   ■■■ offer_letter.pdf.enc
■   ■■■ contract_2024.pdf.enc
■■■ EMP002/
■   ■■■ appraisal_2025.pdf.enc
```

```
■   ■■■ id_proof.jpg.enc
■■■ EMP003/
    ■■■ salary_slip_q4.pdf.enc
```

- All files stored with .enc extension to indicate encrypted state
- S3 server-side encryption (SSE-S3) provides additional layer of protection
- Bucket versioning can be enabled for document history
- S3 lifecycle policies can automate archival to Glacier for compliance

## 5.4 Amazon DynamoDB — Metadata Store

DynamoDB stores structured metadata for every uploaded document, enabling fast search, filtering, and audit capabilities without needing to scan S3 objects directly.

### Table: employee-documents

| Attribute | Type | Description | Example Value |
|---|---|---|---|
| **employee_id** | String (PK) | Partition Key — Employee identifier | EMP001 |
| **file_name** | String (SK) | Sort Key — Original filename | salary_jan2026.txt |
| **document_type** | String | Category of document | Salary Slip |
| **upload_time** | String | ISO 8601 timestamp | 2026-06-06T10:30:00Z |
| **s3_key** | String | Full S3 object path | EMP001/salary_jan2026.txt.enc |

*Figure 5.4 — DynamoDB Table Schema*

# 6 Application Workflow

The application implements three primary workflows: document upload, search, and download. Each workflow is designed to ensure security at every step through AWS service integration.

## 6.1 Upload Workflow

When an employee or HR admin uploads a document, the application orchestrates a multi-step process that ensures the file is encrypted before it ever reaches S3 storage.

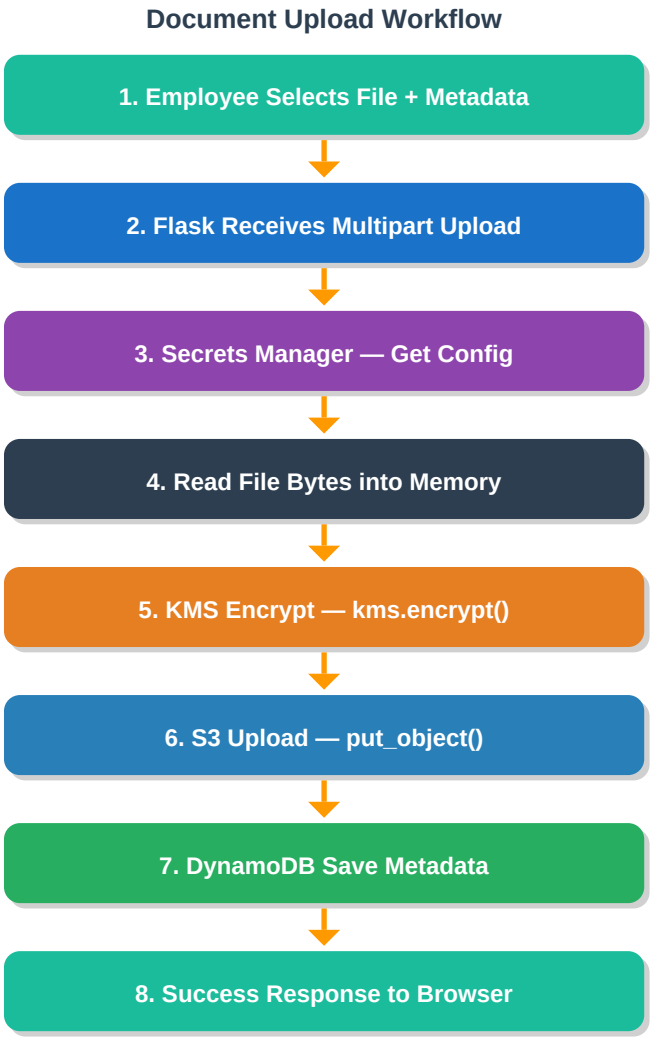


*Figure 6.1 — Document Upload Workflow*

## 6.2 Search Workflow

The search workflow queries DynamoDB using the employee_id as the partition key, returning all document metadata without accessing S3 or performing any decryption operations.

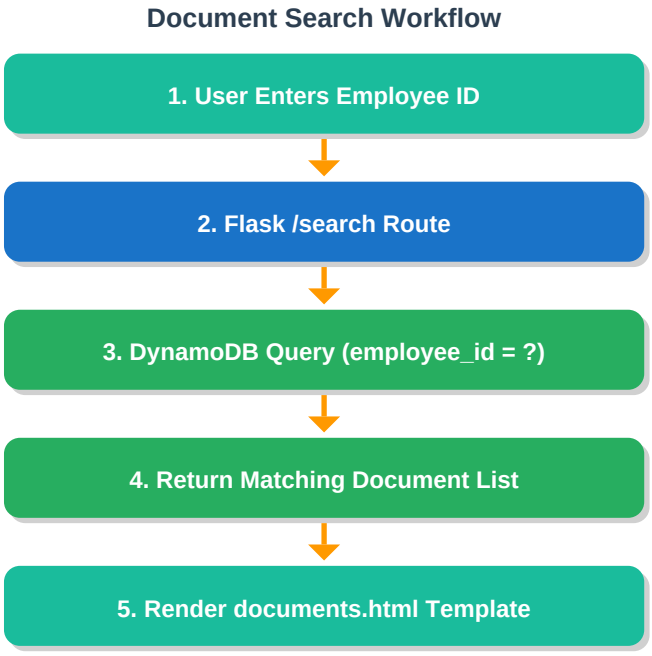


*Figure 6.2 — Document Search Workflow*

## 6.3 Download Workflow

The download workflow retrieves the encrypted file from S3 and decrypts it using KMS before streaming the original file content to the user's browser.

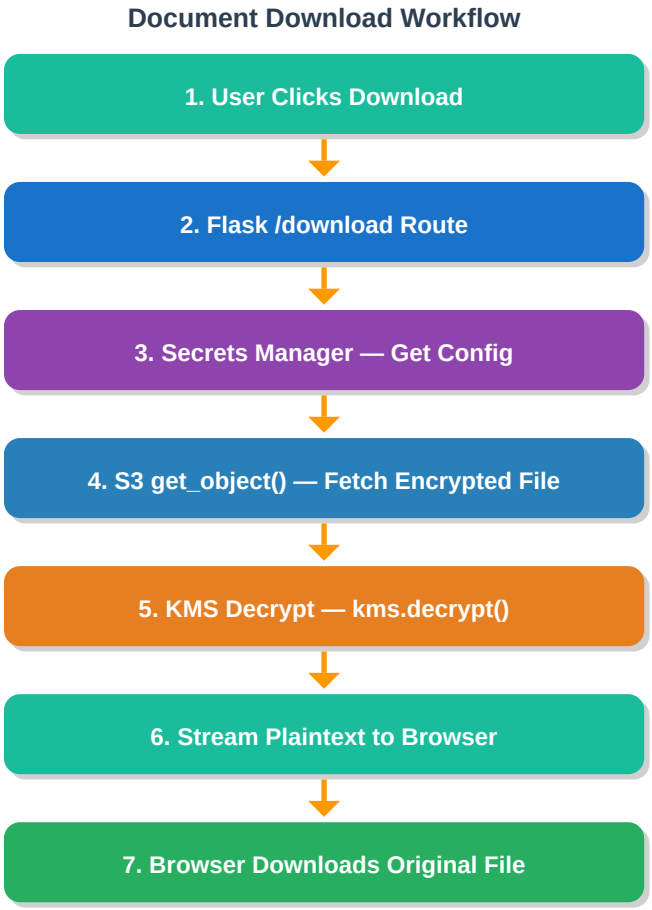


*Figure 6.3 — Document Download Workflow*

# 7 Internal Working & Sequence Diagrams

## 7.1 Upload Sequence Diagram

The following sequence diagram shows the precise order of service interactions during document upload:

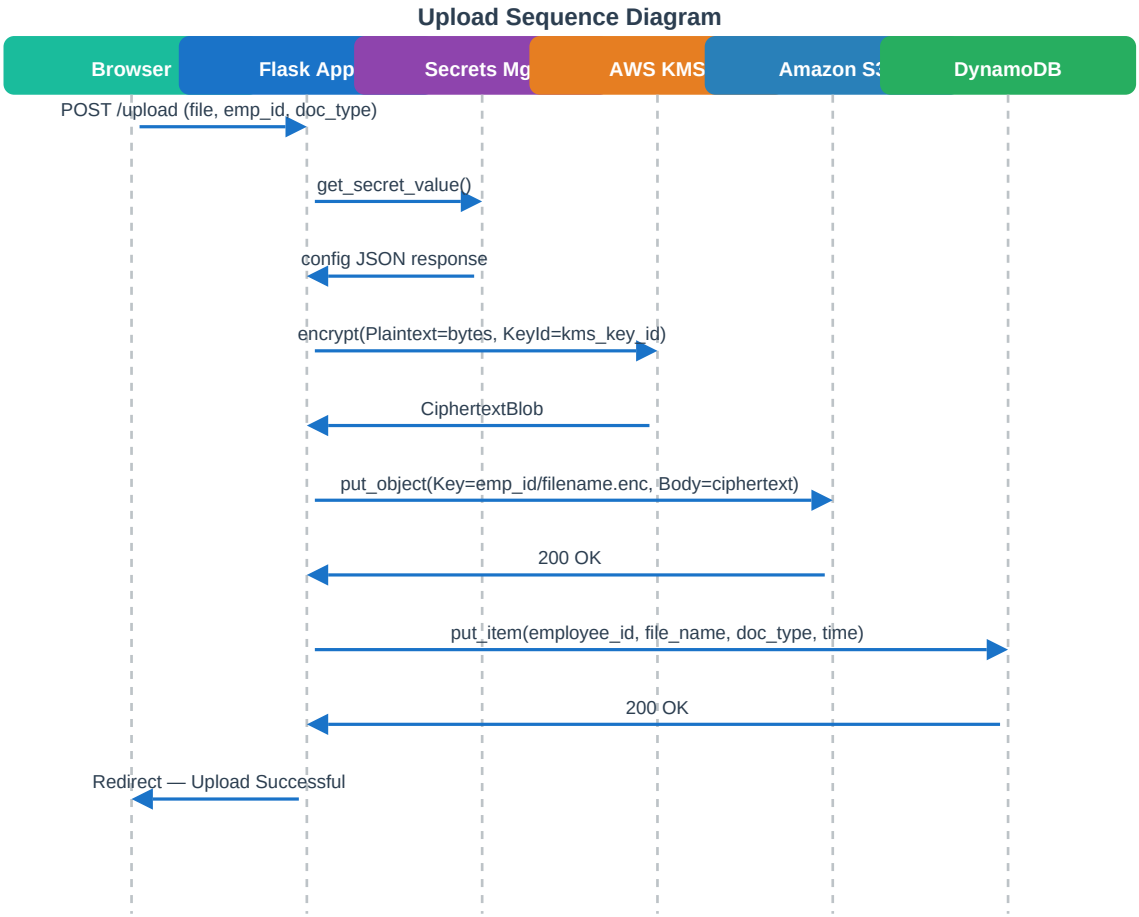


*Figure 7.1 — Upload Sequence Diagram*

## 7.2 Download Sequence Diagram

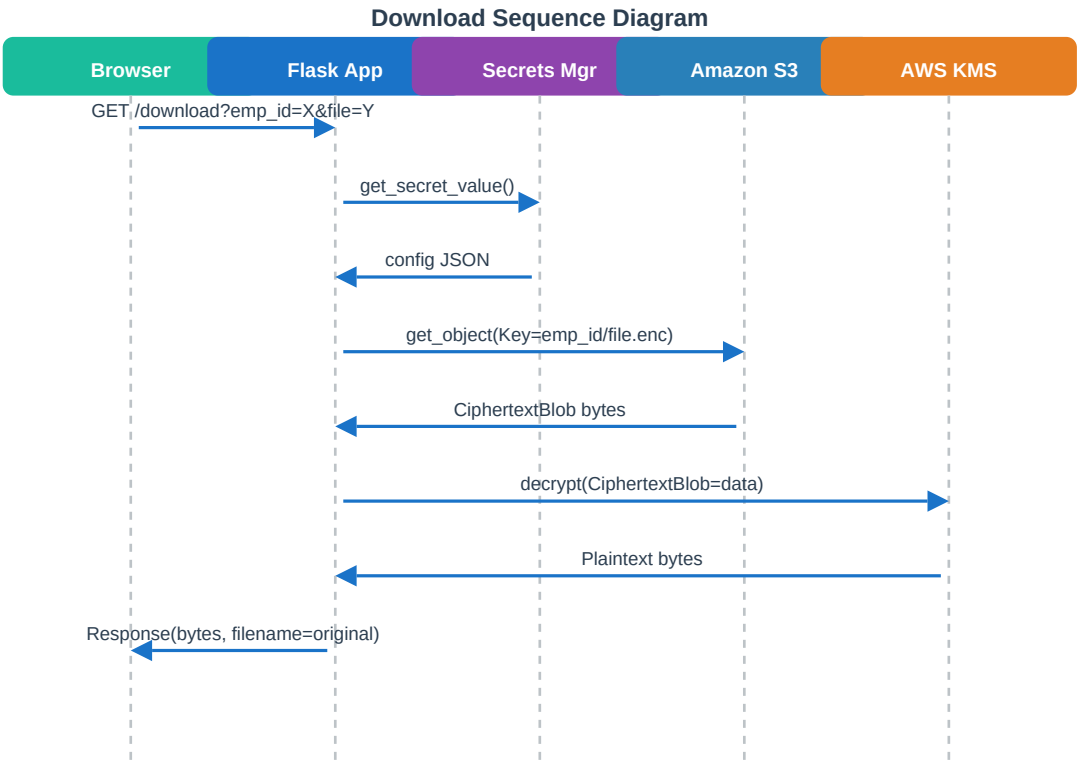


*Figure 7.2 — Download Sequence Diagram*

# 8 Project Structure

The project follows a clean separation of concerns with dedicated service modules for each AWS integration. This structure promotes testability, maintainability, and extensibility.

```
employee-vault/
■
■■■ app.py                    # Flask application entry point
■                             # Route definitions, request handling
■
■■■ services/
■   ■■■ __init__.py
■   ■■■ secret_manager.py     # AWS Secrets Manager integration
■   ■                         # get_secret() → returns config dict
■   ■
■   ■■■ kms_service.py        # AWS KMS encrypt/decrypt operations
■   ■                         # encrypt_file(bytes) → ciphertext
■   ■                         # decrypt_file(ciphertext) → plaintext
■   ■
■   ■■■ s3_service.py         # Amazon S3 upload/download
■   ■                         # upload_file(key, data)
■   ■                         # download_file(key) → bytes
■   ■
■   ■■■ dynamodb_service.py   # DynamoDB CRUD operations
■                             # save_metadata(item)
■                             # get_documents(employee_id)
■
■■■ templates/
■   ■■■ base.html             # Base template with nav/header
■   ■■■ index.html            # Dashboard / home page
■   ■■■ upload.html           # Document upload form
■   ■■■ documents.html        # Search results / document list
■   ■■■ employees.html        # Employee directory
■
■■■ requirements.txt          # Flask, boto3, python-dotenv
■■■ README.md                 # Setup and deployment guide
```

## 8.1 File Responsibilities

| File | Responsibility |
|---|---|
| `app.py` | Flask routes: /, /upload, /search, /download, /employees |
| `secret_manager.py` | Single function: returns parsed config from Secrets Manager |
| `kms_service.py` | encrypt_data(bytes, key_id) and decrypt_data(ciphertext) wrappers |

| | |
|---|---|
| `s3_service.py` | upload_to_s3(bucket, key, data) and download_from_s3(bucket, key) |
| `dynamodb_service.py` | save_document_metadata() and query_documents(employee_id) |

# 9 UI Screen Mockups

The application provides a clean, functional web interface built with Flask's Jinja2 templating engine. The following mockups illustrate the key screens of the application.

## 9.1 Dashboard — Home Screen

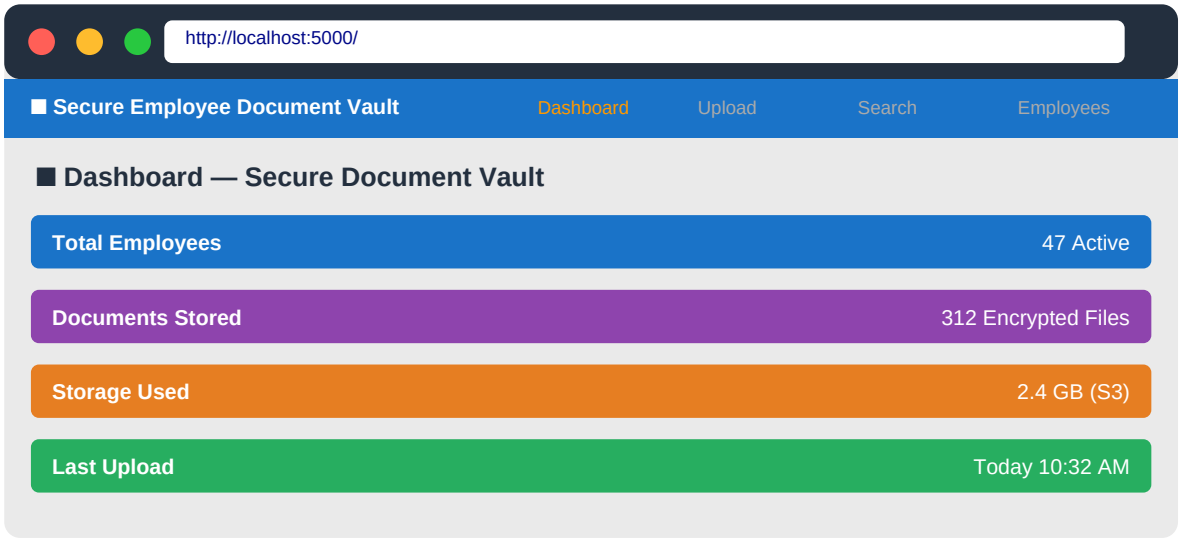


*Figure 9.1 — Dashboard Screen Mockup*

## 9.2 Document Upload Screen

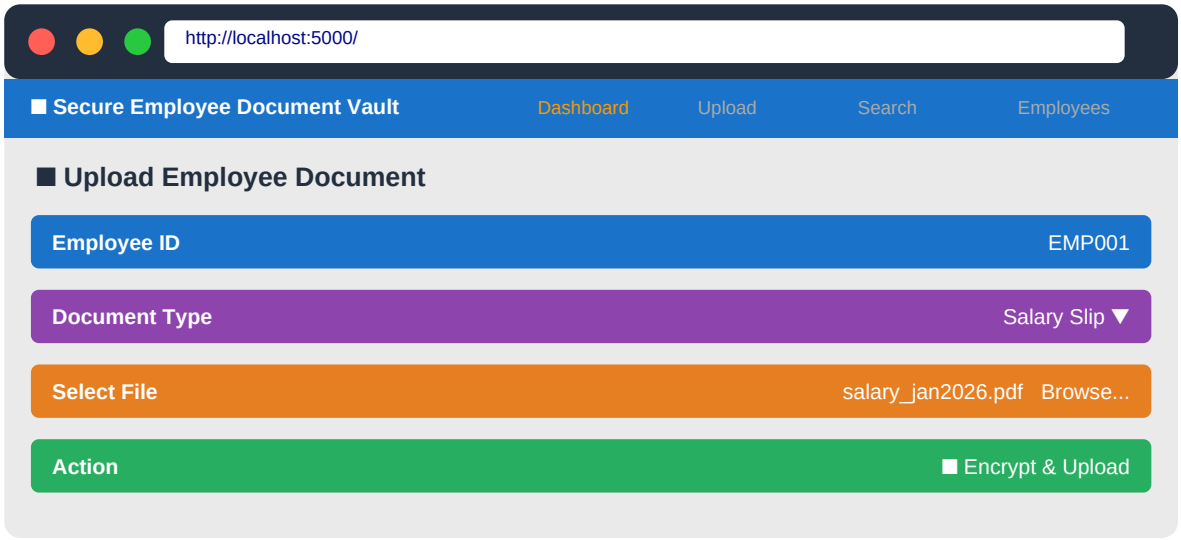


*Figure 9.2 — Document Upload Screen Mockup*

## 9.3 Search & Document List Screen

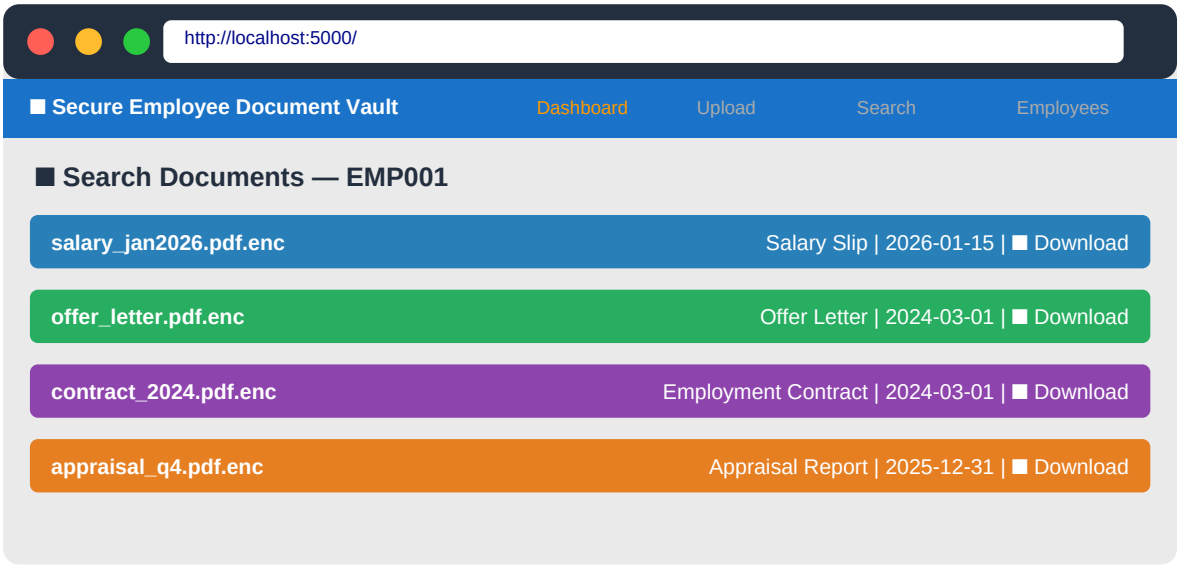


*Figure 9.3 — Search and Document List Screen Mockup*

## 10 Security Features

The application implements defense-in-depth security strategy with multiple independent layers of protection. Each layer addresses a different threat vector, ensuring that the compromise of any single layer does not expose sensitive data.

| | |
|---|---|
| ■ **No Hardcoded Secrets** | Zero credentials in source code. AWS Secrets Manager retrieves all sensitive config at runtime. Git history is clean. |
| ■ **AES-256 Encryption** | AWS KMS encrypts every file with AES-256-GCM before S3 storage. Ciphertext is unreadable without KMS key access. |
| ■ **Encrypted-at-Rest** | Files stored with .enc extension in S3 with additional SSE-S3 layer. Double encryption protection. |
| ■ **Metadata Separation** | Document content (S3) and metadata (DynamoDB) stored in separate services. Metadata breach does not expose file contents. |
| ■ **IAM Role-Based Access** | Boto3 uses EC2/Lambda IAM role credentials. No AWS access keys stored in the application or environment variables. |
| ■ **Audit Trail Ready** | All KMS operations, Secrets Manager accesses, and S3 operations are logged in AWS CloudTrail automatically. |

## 11 Advantages

The Secure Employee Document Vault provides significant advantages over traditional document management approaches, particularly for organizations operating in regulated industries or handling sensitive HR data.

| # | Advantage | Description |
|---|---|---|
| 1 | **Secure Document Storage** | Military-grade AES-256 encryption via AWS KMS ensures documents are secure even if S3 is m |
| 2 | **Cloud-Native Architecture** | Built entirely on AWS managed services — no servers to manage, patch, or scale manually. |
| 3 | **Infinite Scalability** | S3 and DynamoDB scale automatically to handle any volume of documents and employees. |
| 4 | **Centralized Configuration** | Single Secrets Manager secret controls all application configuration. Update once, reflected ever |
| 5 | **AWS Best Practices** | Implements AWS Well-Architected Framework security pillar — IAM roles, secrets rotation, encry |
| 6 | **Easy Maintenance** | Serverless AWS services require zero database administration, patching, or infrastructure manag |
| 7 | **Cost Effective** | Pay-per-use pricing across S3, DynamoDB, KMS, and Secrets Manager. No upfront infrastructur |
| 8 | **Audit Compliance Ready** | CloudTrail logs every service call, S3 access log, and KMS operation for compliance reporting. |

# 12 Current Limitations

The current implementation demonstrates the core architectural concepts of secure document management using AWS services. However, for production deployment at enterprise scale, several enhancements would be recommended.

## 12.1 Direct KMS Encryption Limitation

The current implementation uses direct KMS encryption, where the entire file content is passed to the KMS API. This works well for small files but has a size limitation:

■ AWS KMS encrypt API has a 4KB (4,096 bytes) plaintext size limit. Files larger than 4KB cannot be encrypted directly via KMS API calls in the current implementation.

## 12.2 Current vs Production: Comparison

| Feature | Current Implementation | Production Recommendation |
|---|---|---|
| Encryption Method | Direct KMS Encrypt API | Envelope Encryption |
| File Size Support | ✗ Max 4KB (KMS limit) | ✓ Unlimited file sizes |
| Performance | ✗ KMS API call per file | ✓ Local encrypt, 1 KMS call |
| Cost | ✗ KMS API per operation | ✓ Reduced KMS API costs |
| User Authentication | ✗ Not implemented | ✓ Cognito / JWT Auth |
| Audit Logging | ✗ Manual / CloudTrail only | ✓ Application-level audit logs |
| Role-Based Access | ✗ Not implemented | ✓ Employee vs HR Admin roles |

*Figure 12.1 — Current vs Production Implementation Comparison*

## 12.3 Envelope Encryption — Recommended Solution

Envelope Encryption is the AWS-recommended approach for encrypting large files. Instead of sending the plaintext to KMS, a Data Encryption Key (DEK) is generated and used for local AES encryption, with only the DEK encrypted by KMS.

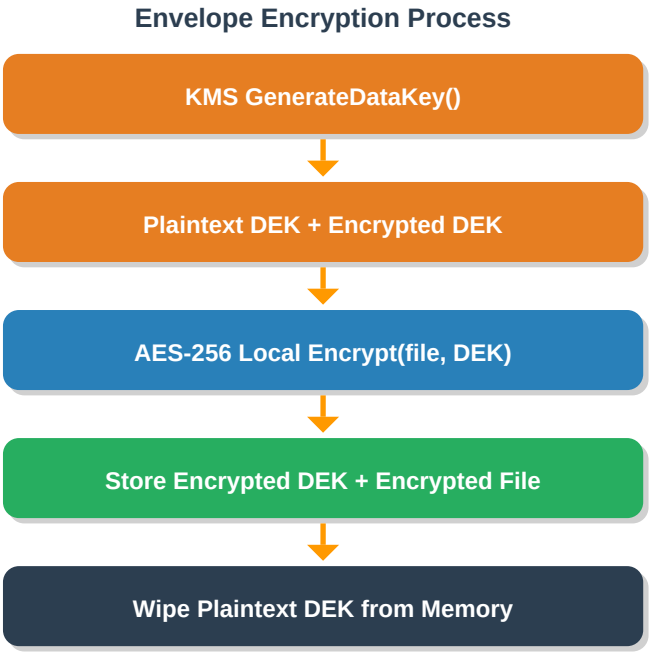


*Figure 12.2 — Envelope Encryption (Production Approach)*

## 13 Future Enhancements

The following roadmap outlines planned enhancements that would evolve this proof-of-concept into a fully production-ready, enterprise-grade document management platform.

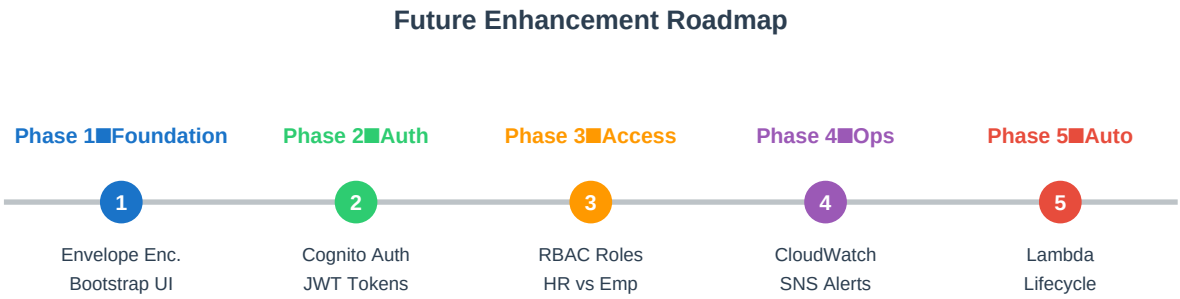


*Figure 13.1 — Future Enhancement Roadmap*

### Phase 1 — Foundation

- Envelope Encryption: Implement GenerateDataKey for unlimited file size support
- Bootstrap 5 UI: Replace plain HTML with responsive, modern dashboard UI
- File type validation: Allow only approved MIME types (PDF, JPG, PNG, DOCX)

### Phase 2 — Authentication

- AWS Cognito Integration: User pools for employee and HR admin authentication
- JWT Token Validation: Secure every API endpoint with token-based auth
- MFA Support: Multi-factor authentication for sensitive operations

### Phase 3 — Access Control

- Role-Based Access Control (RBAC): Employees see only their documents; HR sees all
- Department-level access: Managers can access team documents within scope
- Document-level permissions: Granular access control per document type

### Phase 4 — Operations

- CloudWatch Dashboards: Real-time monitoring of upload/download operations
- SNS Notifications: Email/SMS alerts on document upload or access events
- Audit Log DynamoDB Table: Application-level audit trail separate from CloudTrail

### Phase 5 — Automation

- Lambda Triggers: Auto-thumbnail generation, virus scanning on S3 upload events
- S3 Lifecycle Rules: Automatic archival to Glacier after 90 days

- Automated Secret Rotation: Scheduled Lambda to rotate KMS keys and update Secrets Manager

## 14 Conclusion

The **Secure Employee Document Vault** successfully demonstrates a production-ready approach to secure document management using AWS cloud services and Python Flask. By leveraging the combined power of AWS Secrets Manager, KMS, S3, and DynamoDB, the application addresses the core challenges of enterprise document security.

The project showcases several key software engineering and cloud architecture competencies: the elimination of hardcoded secrets through centralized configuration management, implementation of encryption-at-rest using AWS KMS, structured metadata management via DynamoDB, and a clean service-oriented architecture that separates concerns across independent modules.

From a security perspective, the application follows AWS Well-Architected Framework principles, particularly the Security pillar — using IAM roles rather than static credentials, encrypting data at rest, and maintaining audit trails through AWS CloudTrail. These practices reflect industry standards for handling sensitive PII and confidential HR documentation.

- ✓ Zero hardcoded credentials — AWS Secrets Manager for all configuration
- ✓ AES-256 encryption for every document via AWS KMS
- ✓ Structured metadata enabling fast, serverless document search
- ✓ Service-oriented architecture with clean separation of concerns
- ✓ AWS Well-Architected Security Pillar compliance
- ✓ Production-ready code structure suitable for extension and scale

This project serves as a strong foundation for building enterprise document management platforms and demonstrates practical proficiency in AWS cloud services, Python backend development, cryptographic principles, and secure application design patterns.

## 15 References

| # | Reference | URL |
|---|---|---|
| 1 | AWS Secrets Manager Developer Guide | https://docs.aws.amazon.com/secretsmanager/latest/userguide/intr |
| 2 | AWS Key Management Service Documentation | https://docs.aws.amazon.com/kms/latest/developerguide/overview.h |
| 3 | Amazon S3 Developer Guide | https://docs.aws.amazon.com/AmazonS3/latest/dev/Welcome.html |
| 4 | Amazon DynamoDB Developer Guide | https://docs.aws.amazon.com/amazondynamodb/latest/developerguide |
| 5 | Boto3 Documentation — AWS SDK for Python | https://boto3.amazonaws.com/v1/documentation/api/latest/index.ht |
| 6 | Flask Documentation | https://flask.palletsprojects.com/ |
| 7 | AWS Well-Architected Framework — Security Pillar | https://docs.aws.amazon.com/wellarchitected/latest/security-pill |
| 8 | AWS KMS Envelope Encryption Best Practices | https://docs.aws.amazon.com/kms/latest/developerguide/concepts.h |